

COP2800C Module 6 Graded Programming Assignment

I recommend that you use the Horizon system for this assignment unless you are comfortable working with environment variables and execution search paths. All necessary configurations are already in place on the Horizon system.

Be sure to review the practice exercise for design notes and implementation details used in that exercise that will also apply to this assignment.

For this assignment we will open the data floodgates to release the penguins!

Take a look at the attached palmerpenguins.csv file. This is an excerpt from the actual Palmer data repository which contains hundreds of observations over 7 variables (in other words, a lot of penguins). I have reduced the number of variables to help simplify this assignment. I have also interpolated values missing data since dealing with that is not in the scope of this course (we cover this in the Python course, however; dealing with missing data is a critical operation in data science).

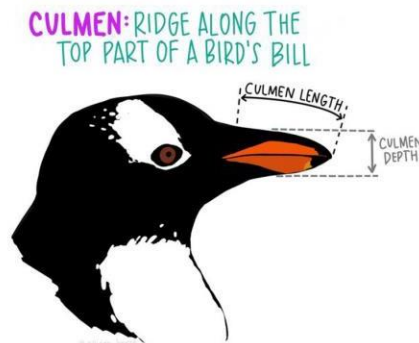
Note that hundreds seems like a lot here, in the world of big data it is really nothing; real-world datasets could have thousands, millions, billions, and even more observations. For a scientific sample-based data set like this, however, a few hundred can provide useful results.

Here are the descriptions of the variables:

- | | |
|---------------------|--|
| 1. SampleNum | unique numeric id of the sample (the penguin) |
| 2. Species | a factor denoting penguin species (Adélie, Chinstrap and Gentoo) |
| 3. CulmenLength_mm | a number denoting bill length (millimeters) |
| 4. CulmenDepth_mm | a number denoting bill depth (millimeters) |
| 5. FlipperLength_mm | an integer denoting flipper length (millimeters) |
| 6. BodyMass_g | an integer denoting body mass (grams) |
| 7. Sex | a factor denoting penguin sex (female, male) |

As mentioned in the description of the "culmen" variables, this is just a fancy word for "bills"; here is an image of what those variables are measuring:

- What is a culmen?



Looking at the variables included in this data, make a guess as to what Java data types can be used to store each of them. Here are the types I came up with:

- | | |
|---------------------|--------------------------------------|
| 1. SampleNum | Integer |
| 2. Species | PenguinSpecies (our enumerated type) |
| 3. CulmenLength_mm | Double |
| 4. CulmenDepth_mm | Double |
| 5. FlipperLength_mm | Double |
| 6. BodyMass_g | Integer |
| 7. Sex | Character (M or F) |

We could use Float instead of Double for the fractional values, but since Double is the default floating point type it can make some operations more efficient (at the expense of a little more memory).

The Sex could be an enumerated type, I was too lazy in my solution to do this. This means we would need to implement some sort of validation to ensure only M or F are allowed.

Here are the instructions for what you need to do for this assignment:

1. Modify all package statements to add another level of packaging named "penguins":

```
package edu.fscj.cop2800c.penguins;
```

2. Include the attached PenguinReader.java file in your package.
3. Download and store the palmerpenguins.csv file at the top level of your project.
4. Make the following changes to the Penguin class from Module 5:

Add the following instance variables to the Penguin class (these correspond to the variables in the data file):

```
private Integer sampleNum = 0;
private PenguinSpecies species = PenguinSpecies.NO_SPECIES;
private Double culmenLen = 0.0,
               culmenDepth = 0.0,
               flipperLen = 0.0;
private Integer bodyMass = 0;
private Character sex = '\0';
```

These instance variables will be read-only for the solution. Create accessors for them but no mutators, and remove the mutator for the species variable.

Modify the overloaded Penguin constructor to accept parameters for these variables and set their value in the constructor using the **this** reference.

5. Modify the PenguinSpecies enum by adding a method:

```
public static PenguinSpecies parseSpecies(String speciesStr)
```

This method will parse a string containing a species name and return the corresponding PenguinSpecies enum value (use the methods in the BoxSize and Color enums in the Module 6 practice exercise solution as a reference).

6. Using the Module 6 practice exercise solution's BoxFactory class as a reference, make the following changes to the PenguinRookery class:

Remove the TOTAL_SPECIES constant and add a constant for the maximum number of penguins we will allow, e.g.

```
public static final int MAX_PENGUINS = 1000;
```

Modify the toString() method so it returns the contents of the rookery. No special formatting is necessary for this data.

Add a table method to the PenguinRookery that returns a formatted version of the data (see Expected Output, below).

Remove the code in the PenguinRookery class's main method that deals with individual penguins and instead call the PenguinReader.read() method; then process the returned array to create a local array of penguins.

Here are the parse operations in the for loop that I used for my instance variables; the second dimension of the String[][] array returned by the read method corresponds to the variable's position in the array. The final statement extracts the first character from the "MALE/FEMALE" string.

```
Integer newSampleNum = Integer.parseInt(inputArray[rows][0]);
PenguinSpecies newSpecies =
    PenguinSpecies.parseSpecies(inputArray[rows][1]);
Double newCulmenLen = Double.parseDouble(inputArray[rows][2]);
Double newCulmenDepth = Double.parseDouble(inputArray[rows][3]);
Double newFlipperLen = Double.parseDouble(inputArray[rows][4]);
Integer newBodyMass = Integer.parseInt(inputArray[rows][5]);
Character newSex = inputArray[rows][6].charAt(0);
```

Modify the PenguinRookery's overloaded constructor to accept an int parameter which represents the number of elements to allocate for the rookery's array. If you have reviewed the palmerpenguins.csv file you will have seen that we will not read the maximum number of Penguins from that file, so we will be creating a partially-filled array in the main method since the local array in the main method is allocated with the maximum number of Penguins. After we have read in the Penguin data we will know the number of elements necessary, that's the value we pass to the PenguinRookery constructor. Here's the header for that constructor in my solution:

```
public PenguinRookery(int birdCount, Penguin[] penguins) {
```

You'll need to allocate the array instance variable and copy each element from the parameter array into the array instance variable.

After extracting these values, create a new Penguin using the modified constructor overload and add it to the local array:

```
penguinArray[birdCount++] = new Penguin(newSampleNum, newSpecies,  
                                         newCulmenLen, newCulmenDepth,  
                                         newFlipperLen, newBodyMass,  
                                         newSex);
```

Note: **penguinArray** was declared at the top of the main method like this:

```
Penguin[] penguinArray = new Penguin[MAX_PENGUINS];
```

Call the table method as the last action in this main method and print the returned string (again, refer to Expected Output, below).

7. Create a JAR file for your PenguinRookery application which contains your **source code and class files** for all four classes **as well as the .csv data file**. I should be able to execute your code from the JAR file without extracting it, but I also will extract your source code to view and grade it.

8. Attach and submit your JAR file (you do not need to submit your source code files since your JAR file will contain them). **Be sure to include the .csv file at the top level of the project in the JAR file so I can access it if needed.**

Expected output is as follows (not all rows are displayed)

Palmer Penguin Rookery:

SampleNum	Species	CulmenLen	CulmenDepth	FlipperLen	BodyMass	Sex
1	ADELIE	39.1	18.7	181.0	3750	M
2	ADELIE	39.5	17.4	186.0	3800	F
3	ADELIE	40.3	18.0	195.0	3250	F
4	ADELIE	36.7	19.3	193.0	3450	F
...						
339	GENTOO	46.8	14.3	215.0	4850	F
340	GENTOO	50.4	15.7	222.0	5750	M
341	GENTOO	45.2	14.8	212.0	5200	F
342	GENTOO	49.9	16.1	213.0	5400	M